

QUEUE

자료구조 스터디 Week4

학생회



Made with Gamma

INDEX

01

THEORY

02

IMPLEMENTATION

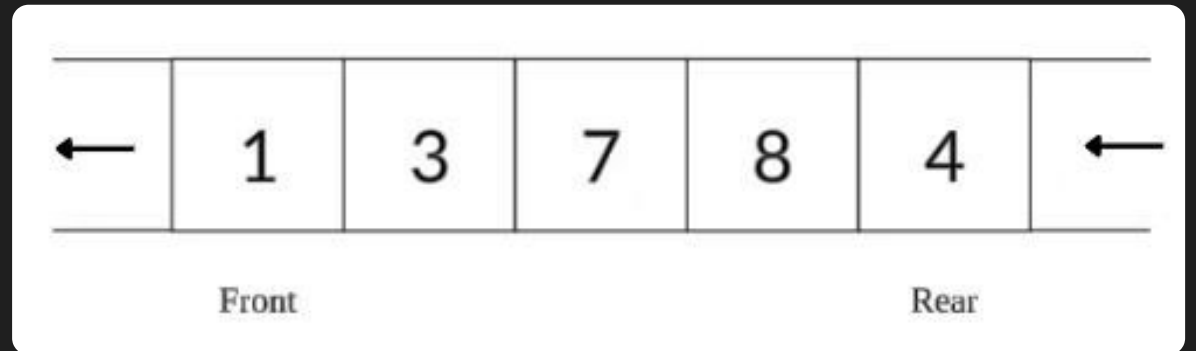
03

PROBLEM SOLVING

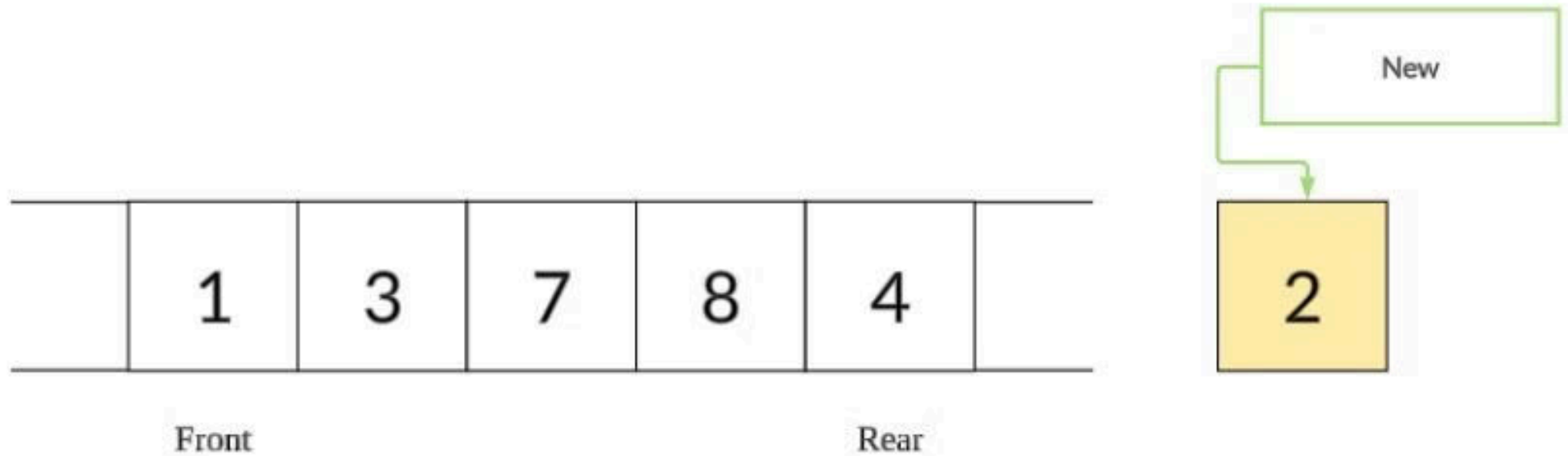
큐

큐(Queue)는 선입선출(FIFO, First In First Out)의 자료구조로, 먼저 들어온 데이터가 먼저 나가는 구조를 가지고 있습니다.

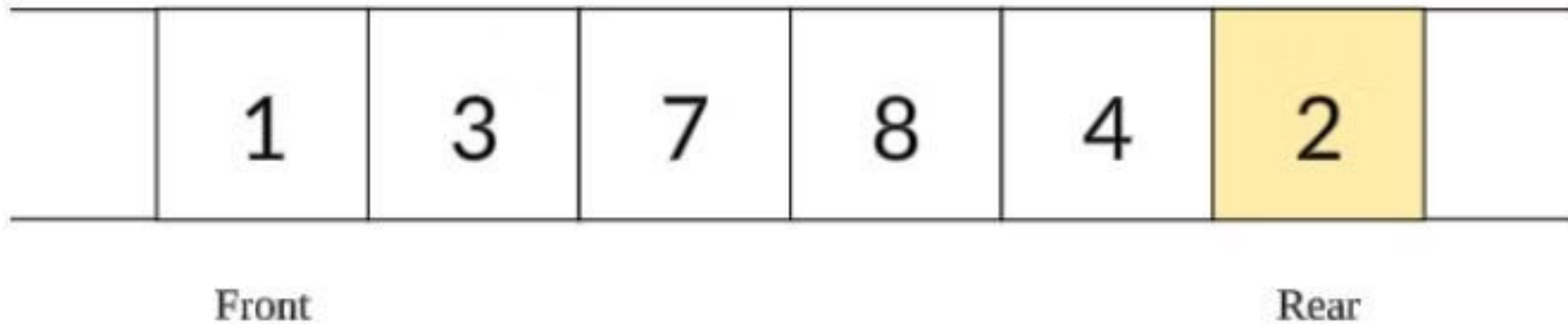
삽입: 큐의 뒤에 추가 / 삭제: 큐의 앞에서 제거



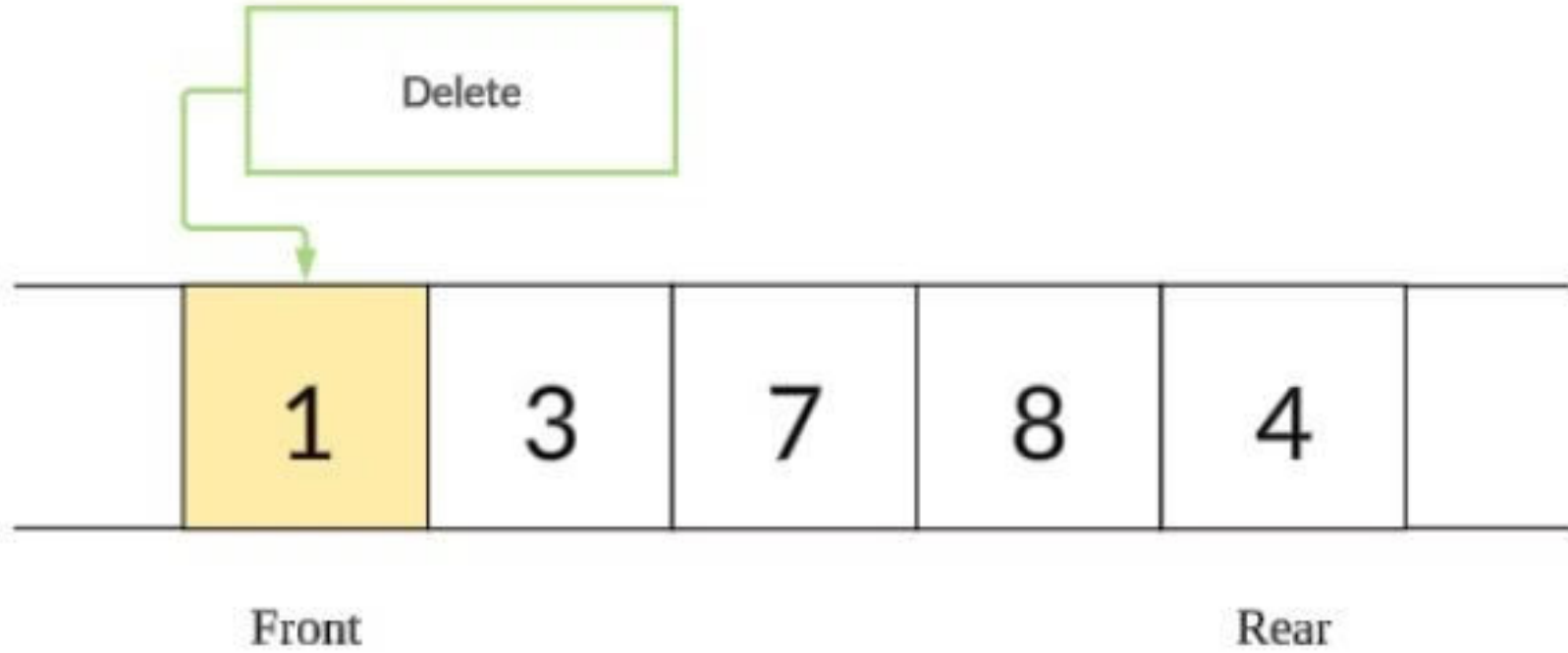
큐의 이해 - 삽입(큐의 뒤에 추가)



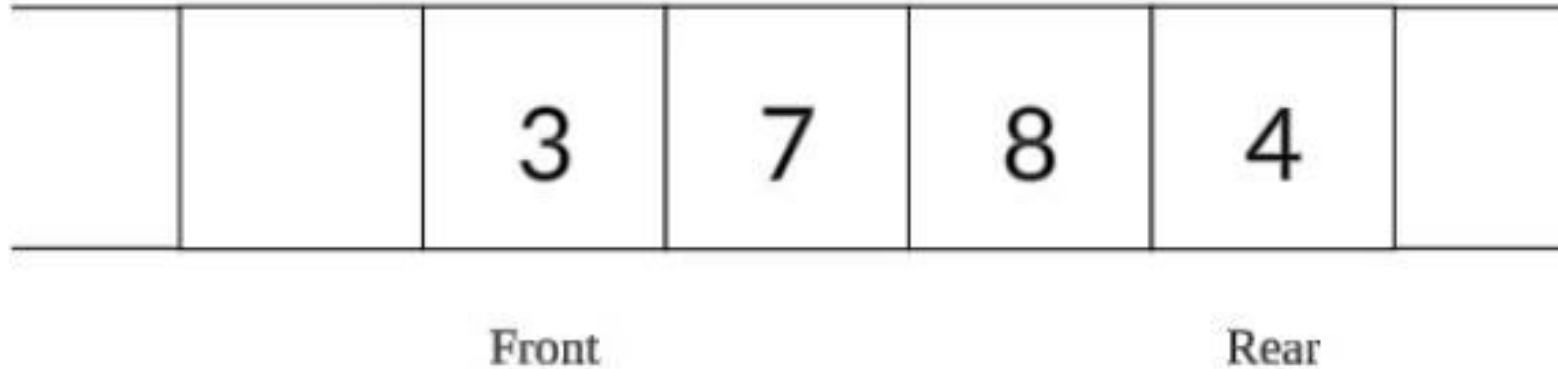
큐의 이해 - 삽입(큐의 뒤에 추가)



큐의 이해 - 삭제(큐의 앞에서 제거)



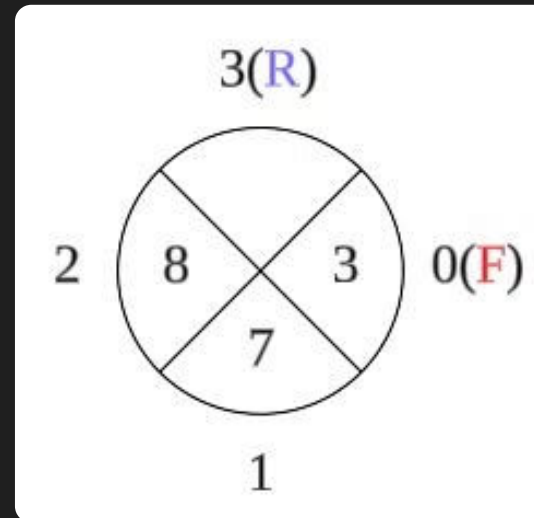
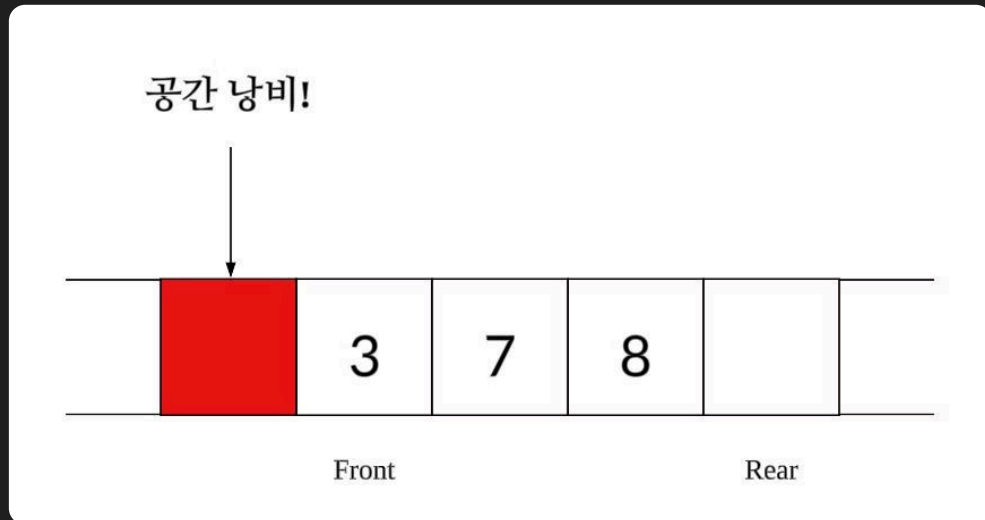
큐의 이해 - 삭제(큐의 앞에서 제거)



문제점

배열로 큐 구현 시 삭제를 할 때마다 공간이 낭비됨 → 배열로 큐 구현 시 환형(원형) 큐 구현

(링크드 리스트는 노드 자체를 삭제하기 때문에 공간이 낭비되지 않음 → 선형 큐 구현)



IMPLEMENTATION



큐의 구현 방법

1

배열

배열 기반 환형 큐

2

링크드 리스트

링크드 리스트 기반 선형 큐

큐의 기본 연산

1 isEmpty

큐가 비어 있는지 확인

2 Size

큐의 크기 조회

3 Front

제일 앞 요소를 조회

4 Rear

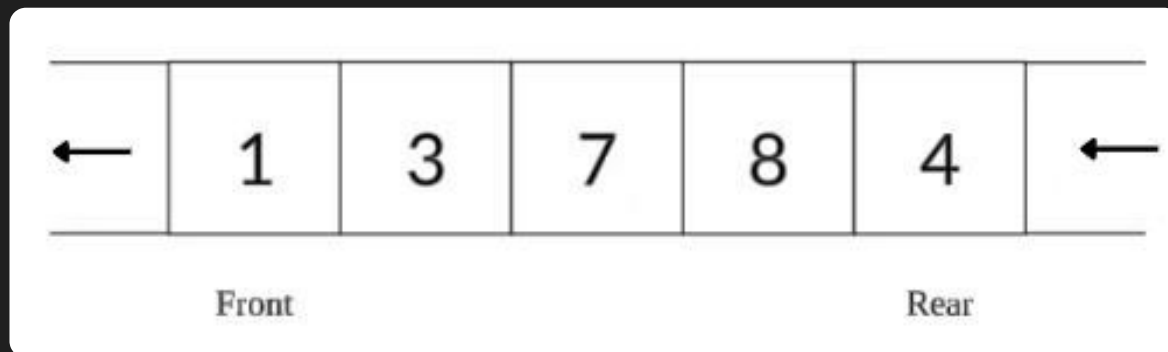
제일 뒤 요소를 조회

5 Enqueue

제일 뒤에 요소를 추가

6 Dequeue

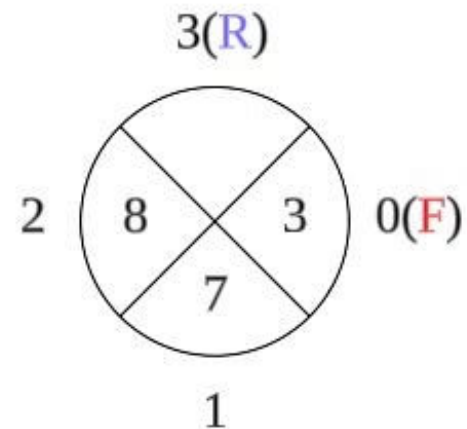
제일 앞의 요소를 제거



ARRAY_QUEUE

멤버 변수

```
class arrayQueue {  
private:  
    int* arr;        // 배열  
    int capacity;    // 큐의 최대 크기  
    int f;           // front의 index  
    int r;           // rear의 index + 1  
    int s;           // 큐의 크기
```



생성자

```
public:
    arrayQueue(int capacity) {
        this->capacity = capacity;
        arr = new int[capacity];
        f = r = 0;
        s = 0;
    }
```

- this→capacity: arrayQueue 클래스의 멤버변수
- int* arr = new int[capacity]
- f와 r 모두 0으로 초기화
- size 0으로 초기화

isEmpty

```
bool empty() {  
    return (s == 0);  
}
```

- size가 0이라면 True 리턴
- size가 0이 아니라면 False 리턴

size

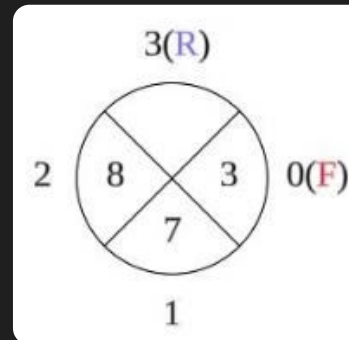
```
int size() {  
    return s;  
}
```

- size 리턴

Front

```
void front() {  
    if (empty()) {  
        cout << "Empty\n";  
        return;  
    }  
  
    cout << arr[f] << '\n';  
}
```

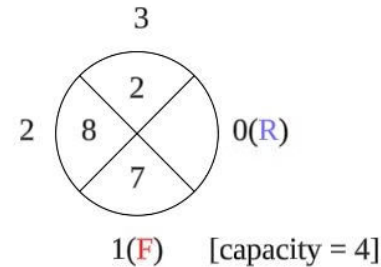
- 문제에 제시된 예외처리
- 가장 앞 원소의 index는 f



Rear

```
void rear() {  
    if (empty()) {  
        cout << "Empty\n";  
        return;  
    }  
  
    if (r == 0) {  
        cout << arr[capacity - 1] << '\n';  
        return;  
    }  
  
    cout << arr[r - 1] << '\n';  
}
```

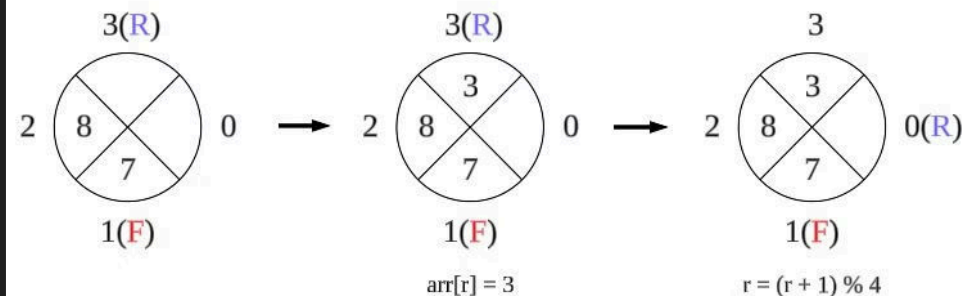
- 문제에 제시된 예외처리
- 가장 뒤 원소의 index는 $r - 1$
- $r = 0$ 이라면 런타임에러 발생
- $r = 0$ 이라면 마지막 원소는 $\text{capacity} - 1$



Enqueue

```
void enqueue(int value) {  
    if (full()) {  
        cout << "Full\n";  
        return;  
    }  
  
    arr[r] = value;  
  
    r = (r + 1) % capacity;  
    s++;  
}
```

- 문제에 제시된 예외처리

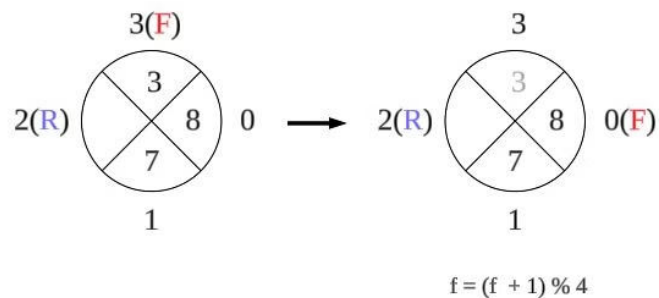


- size 1 증가

Deque

```
void dequeue() {  
    if (empty()) {  
        cout << "Empty\n";  
        return;  
    }  
  
    cout << arr[f] << '\n';  
  
    f = (f + 1) % capacity;  
    s--;  
}
```

- 문제에 제시된 예외처리
- 문제에 제시된 동작 수행



- size 1 감소

LIST_QUEUE

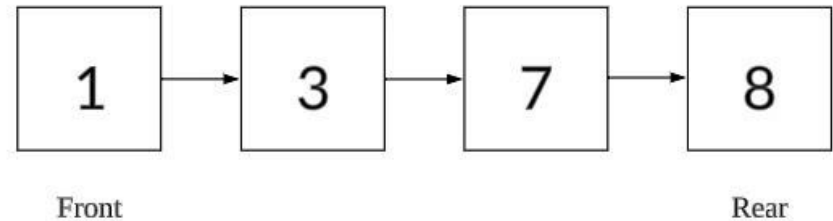
노드 클래스

```
class node {  
private:  
    int value;  
    node* next;  
  
public:  
    node(int value) {  
        this->value = value;  
        next = nullptr;  
    }  
  
    friend class listQueue;  
};
```

- 가장 앞, 가장 뒤 원소만 처리하기에 싱글 링리
- value 초기화, next는 nullptr 초기화
- friend class 선언 → 링리에서 노드 편하게 사용

멤버 변수

```
class listQueue {  
private:  
    node* f;    // front(head)  
    node* r;    // rear(tail)  
    int s;      // 큐의 크기
```



생성자

```
public:
    listQueue() {
        f = r = nullptr;
        s = 0;
    }
```

- f와 r 모두 0으로 초기화
- size 0으로 초기화

isEmpty

```
bool empty() {  
    return (s == 0);  
}
```

- size가 0이라면 True 리턴
- size가 0이 아니라면 False 리턴

size

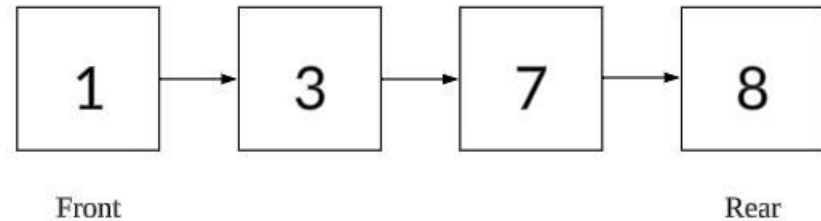
```
int size() {  
    return s;  
}
```

- size 리턴

Front

```
void front() {  
    if (empty()) {  
        cout << "Empty\n";  
        return;  
    }  
  
    cout << f->value << '\n';  
}
```

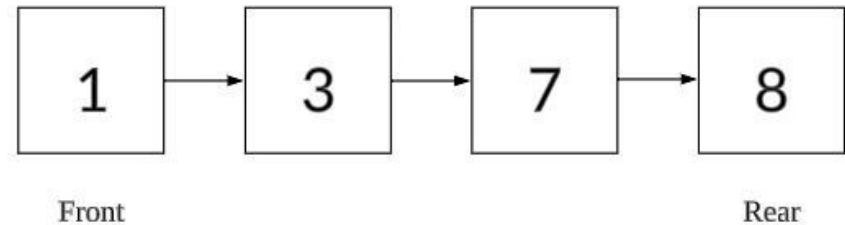
- 문제에 제시된 예외처리
- Front의 값 출력



Rear

```
void rear() {  
    if (empty()) {  
        cout << "Empty\n";  
        return;  
    }  
  
    cout << r->value << '\n';  
}
```

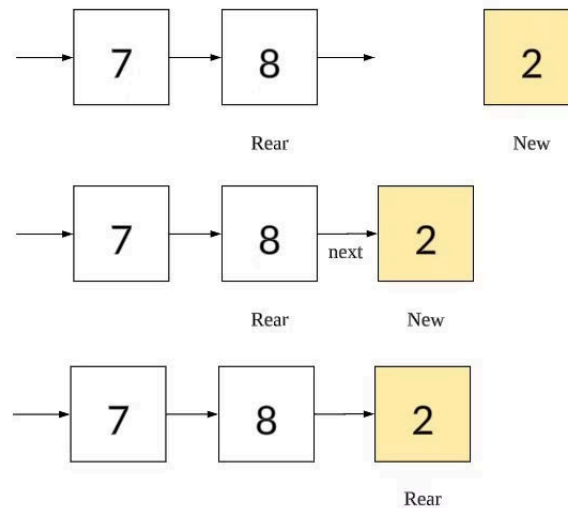
- 문제에 제시된 예외처리
- Rear의 값 출력



Enqueue

```
void enqueue(int value) {  
    node* new_node = new node(value);  
  
    if (empty()) {  
        f = r = new_node;    // 예외  
    }  
  
    else {  
        r->next = new_node;  
        r = new_node;  
    }  
  
    s++;  
}
```

- 삽입할 노드 생성

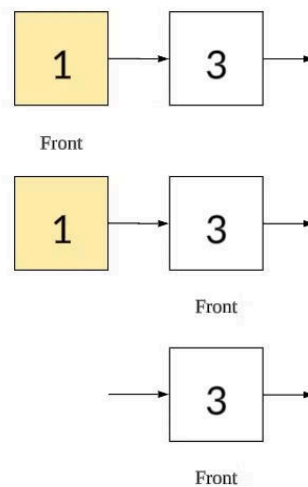


- size 1 증가

Dequeue

```
void dequeue() {  
    if (empty()) {  
        cout << "Empty\n";  
        return;  
    }  
  
    node* del_node = f;  
  
    if (s == 1) {  
        f = r = nullptr;    // 예외  
    }  
  
    else {  
        f = f->next;  
    }  
  
    cout << del_node->value << '\n';  
    // del_node->next = nullptr;  
    delete del_node;  
    s--;  
}
```

- 문제에 제시된 예외처리
- 삭제할 노드 = front



- size 1 감소

PROBLEM SOLVING

**THANK'S FOR
WATCHING**